Chapter 6: XGBoost Hyperparameters

XGBoost has many hyperparameters. XGBoost base learner hyperparameters incorporate all decision tree hyperparameters as a starting point. There are gradient boosting hyperparameters, since XGBoost is an enhanced version of gradient boosting. Hyperparameters unique to XGBoost are designed to improve upon accuracy and speed. However, trying to tackle all XGBoost hyperparameters at once can be dizzying.

In ***Chapter 2***, *Decision Trees in Depth*, we reviewed and applied base learner hyperparameters such as `max_depth`, while in ***Chapter 4***, *From Gradient Boosting to XGBoost*, we applied important XGBoost hyperparameters, including `n_estimators` and `learning_rate`. We will revisit these hyperparameters in this chapter in the context of XGBoost. Additionally, we will also learn about novel XGBoost hyperparameters such as `gamma` and a technique called **early stopping**.

In this chapter, to gain proficiency in fine-tuning XGBoost hyperparameters, we will cover the following main topics:

- Preparing data and base models
- Tuning core XGBoost hyperparameters
- Applying early stopping
- Putting it all together

Technical requirements

The code for this chapter can be found at **https://github.com/PacktPublishing/Hands-On-Gradient-Boosting-with-XGBoost-and-Scikit-learn/tree/master/Chapter06**.

Preparing data and base models

Before introducing and applying XGBoost hyperparameters, let's prepare by doing the following:

- Getting the **heart disease dataset**
- Building an `XGBClassifier` model
- Implementing `StratifiedKFold`
- Scoring a **baseline XGBoost model**
- Combining `GridSearchCV` with `RandomizedSearchCV` to form one powerful function

Good preparation is essential for gaining accuracy, consistency, and speed when fine-tuning hyperparameters.

The heart disease dataset

The dataset used throughout this chapter is the heart disease dataset originally presented in ***Chapter 2***, *Decision Trees in Depth*. We have chosen the same dataset to maximize the time spent doing hyperparameter fine-tuning, and to minimize the time spent on data analysis. Let's begin the process:

1. Go to **https://github.com/PacktPublishing/Hands-On-Gradient-Boosting-with-XGBoost-and-Scikit-learn/tree/master/Chapter06** to load `heart_disease.csv` into a DataFrame and display the first five rows. Here is the code:

```
import pandas as pd
df = pd.read_csv('heart_disease.csv')
df.head()
```

The result should look as follows:

	age	sex	cp	trestbps	chol	fbs	restecg	thalach	exang	oldpeak	slope	ca	thal	target
0	63	1	3	145	233	1	0	150	0	2.3	0	0	1	1
1	37	1	2	130	250	0	1	187	0	3.5	0	0	2	1
2	41	0	1	130	204	0	0	172	0	1.4	2	0	2	1
3	56	1	1	120	236	0	1	178	0	0.8	2	0	2	1
4	57	0	0	120	354	0	1	163	1	0.6	2	0	2	1

Figure 6.1 – The first five rows

The last column, **target**, is the target column, where **1** indicates presence, meaning the patient has a heart disease, and **2** indicates absence. For detailed information on the other columns, visit **https://archive.ics.uci.edu/ml/datasets/Heart+Disease** at the UCI Machine Learning Repository, or see ***Chapter 2***, *Decision Trees in Depth*.

2. Now, check `df.info()` to ensure that the data is all numerical with no null values:

```
df.info()
```

Here is the output:

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 303 entries, 0 to 302
Data columns (total 14 columns):
#   Column    Non-Null Count  Dtype
---  ------    --------------  -----
0   age       303 non-null    int64
1   sex       303 non-null    int64
2   cp        303 non-null    int64
3   trestbps  303 non-null    int64
4   chol      303 non-null    int64
5   fbs       303 non-null    int64
6   restecg   303 non-null    int64
7   thalach   303 non-null    int64
8   exang     303 non-null    int64
9   oldpeak   303 non-null    float64
```

```
10  slope     303 non-null    int64
11  ca        303 non-null    int64
12  thal      303 non-null    int64
13  target    303 non-null    int64
dtypes: float64(1), int64(13)
memory usage: 33.3 KB
```

Since all data points are non-null and numerical, the data is machine learning-ready. It's time to build a classifier.

XGBClassifier

Before tuning hyperparameters, let's build a classifier so that we can obtain a baseline score as a starting point.

To build an XGBoost classifier, follow these steps:

1. Download **`XGBClassifier`** and **`accuracy_score`** from their respective libraries. The code is as follows:
   ```
   from xgboost import XGBClassifier
   from sklearn.metrics import accuracy_score
   ```
2. Declare **`X`** as the predictor columns and **`y`** as the target column, where the last row is the target column:
   ```
   X = df.iloc[:, :-1]
   y = df.iloc[:, -1]
   ```
3. Initialize **`XGBClassifier`** with the **`booster='gbtree'`** and **`objective='binary:logistic'`** defaults along with **`random_state=2`**:
   ```
   model = XGBClassifier(booster='gbtree', objective='binary:logistic',
   random_state=2)
   ```

The **`'gbtree'`** booster, the base learner, is a gradient boosted tree.

The **`'binary:logistic'`** objective is standard for binary classification in determining the loss function. Although **`XGBClassifier`** includes these values by default, we include them here to gain familiarity in preparation of modifying them in later chapters.

4. To score the baseline model, import **`cross_val_score`** and **`numpy`** to fit, score, and display results:
   ```
   from sklearn.model_selection import cross_val_score
   import numpy as np
   scores = cross_val_score(model, X, y, cv=5)
   print('Accuracy:', np.round(scores, 2))
   print('Accuracy mean: %0.2f' % (scores.mean()))
   ```

The accuracy score is as follows:

```
Accuracy: [0.85 0.85 0.77 0.78 0.77]
Accuracy mean: 0.81
```

An accuracy score of 81% is an excellent starting point, considerably higher than the 76% cross-validation obtained by **DecisionTreeClassifier** in ***Chapter 2***, *Decision Trees in Depth*.

We used **cross_val_score** here, and we will use **GridSearchCV** to tune hyperparameters. Next, let's find a way to ensure that the test folds are the same using **StratifiedKFold**.

StratifiedKFold

When fine-tuning hyperparameters, **GridSearchCV** and **RandomizedSearchCV** are the standard options. An issue from ***Chapter 2***, *Decision Trees in Depth*, is that **cross_val_score** and **GridSearchCV**/**RandomizedSearchCV** do not split data the same way.

One solution is to use **StratifiedKFold** whenever cross-validation is used.

A stratified fold includes the same percentage of target values in each fold. If a dataset contains 60% 1s and 40% 0s in the target column, each stratified test set contains 60% 1s and 40% 0s. When folds are random, it's possible that one test set contains a 70-30 split while another contains a 50-50 split of target values.

Tip

When using **train_test_split**, *the shuffle and stratify parameters use defaults to stratify the data for you. See* **https://scikit-learn.org/stable/modules/generated/sklearn.model_selection.train_test_split.html** *for general information.*

To use **StratifiedKFold**, do the following:

1. Implement **StratifiedKFold** from **sklearn.model_selection**:
   ```
   from sklearn.model_selection import StratifiedKFold
   ```
2. Next, define the number of folds as **kfold** by selecting **n_splits=5**, **shuffle=True**, and **random_state=2** as the **StratifiedKFold** parameters. Note that **random_state** provides a consistent ordering of indices, while **shuffle=True** allows rows to be initially shuffled:
   ```
   kfold = StratifiedKFold(n_splits=5, shuffle=True, random_state=2)
   ```

The **kfold** variable can now be used inside **cross_val_score**, **GridSeachCV**, and **RandomizedSearchCV** to ensure consistent results.

Now, let's return to **cross_val_score** using **kfold** so that we have an appropriate baseline for comparison.

Baseline model

Now that we have a method for obtaining consistent folds, it's time to score an official baseline model using **`cv=kfold`** inside **`cross_val_score`**. The code is as follows:

```
scores = cross_val_score(model, X, y, cv=kfold)
print('Accuracy:', np.round(scores, 2))
print('Accuracy mean: %0.2f' % (scores.mean()))
```

The accuracy score is as follows:

```
Accuracy: [0.72 0.82 0.75 0.8 0.82]
Accuracy mean: 0.78
```

The score has gone down. What does this mean?

It's important not to become too invested in obtaining the highest possible score. In this case, we trained the same **`XGBClassifier`** model on different folds and obtained different scores. This shows the importance of being consistent with test folds when training models, and why the score is not necessarily the most important thing. Although when choosing between models, obtaining the best possible score is an optimal strategy, the difference in scores here reveals that the model is not necessarily better. In this case, the two models have the same hyperparameters, and the difference in scores is attributed to the different folds.

The point here is to use the same folds to obtain new scores when fine-tuning hyperparameters with **`GridSearchCV`** and **`RandomizedSearchCV`** so that the comparison of scores is fair.

Combining GridSearchCV and RandomizedSearchCV

`GridSearchCV` searches all possible combinations in a hyperparameter grid to find the best results. **`RandomizedSearchCV`** selects 10 random hyperparameter combinations by default. **`RandomizedSearchCV`** is typically used when **`GridSearchCV`** becomes unwieldy because there are too many hyperparameter combinations to exhaustively check each one.

Instead of writing two separate functions for **`GridSearchCV`** and **`RandomizedSearchCV`**, we will combine them into one streamlined function with the following steps:

1. Import **`GridSearchCV`** and **`RandomizedSearchCV`** from **`sklearn.model_selection`**:
   `from sklearn.model_selection import GridSearchCV, RandomizedSearchCV`
2. Define a **`grid_search`** function with the **`params`** dictionary as input, along with **`random=False`**:
   `def grid_search(params, random=False):`
3. Initialize an XGBoost classifier using the standard defaults:
   `xgb = XGBClassifier(booster='gbtree', objective='binary:logistic', random_state=2)`
4. If **`random=True`**, initialize **`RandomizedSearchCV`** with **`xgb`** and the **`params`** dictionary. Set **`n_iter=20`** to allow 20 random combinations instead of

10. Otherwise, initialize **`GridSearchCV`** with the same inputs. Make sure to set **`cv=kfold`** for consistent results:

```
    if random:
        grid = RandomizedSearchCV(xgb, params, cv=kfold, n_iter=20,
n_jobs=-1)
    else:
        grid = GridSearchCV(xgb, params, cv=kfold, n_jobs=-1)
```

5. Fit **`X`** and **`y`** to the **`grid`** model:

```
grid.fit(X, y)
```

6. Obtain and print **`best_params_`**:

```
best_params = grid.best_params_
print("Best params:", best_params)
```

7. Obtain and print **`best_score_`**:

```
best_score = grid.best_score_
print("Training score: {:.3f}".format(best_score))
```

The **`grid_search`** function can now be used to fine-tune all hyperparameters.

Tuning XGBoost hyperparameters

There are many XGBoost hyperparameters, some of which have been introduced in previous chapters. The following table summarizes key XGBoost hyperparameters, most of which we cover in this book.

Note

The XGBoost hyperparameters presented here are not meant to be exhaustive, but they are meant to be comprehensive. For a complete list of hyperparameters, read the official documentation, XGBoost Parameters, at **https://xgboost.readthedocs.io/en/latest/parameter.html**.

Following the table, further explanations and examples are provided:

Name	Default	Range	Effect	Notes/Tips
n_estimators	100	[1, inf)	Increasing may improve scores with large data.	The number of trees in the ensemble.
learning_rate **alias:eta**	0.3	[0, inf)	Decreasing prevents overfitting.	Shrinks the tree weights in each round of boosting.
max_depth	6	[0, inf)	Decreasing prevents overfitting.	The depth of the tree. 0 is an option in a loss-guided growing policy.
gamma **alias: min_split_loss**	0	[0, inf)	Increasing prevents overfitting.	Low values, usually lower than 10, are standard.
min_child_weight	1	[0, inf)	Increasing prevents overfitting.	The minimum sum of weights required for a node to split.
subsample	1	(0, 1]	Decreasing prevents overfitting.	Limits the percentage of training rows for each boosting round.
colsample_bytree	1	(0, 1]	Decreasing prevents overfitting.	Limits the percentage of training columns for each boosting round.
colsample_bylevel	1	(0, 1]	Decreasing prevents overfitting.	Limits the percentage of columns for each depth level of the tree.
colsample_bynode	1	(0, 1]	Decreasing prevents overfitting.	Limits the percentage of columns to evaluate splits.
scale_pos_weight	1	(0, inf)	Sum(negatives)/ Sum(positives) balances data.	Used for imbalanced datasets. See *Chapter 5*, *XGBoost Unveiled*, and *Chapter 7*, *Discovering Exoplanets with XGBoost*.
max_delta_step	0	[0, inf)	Increasing prevents overfitting.	Only recommended for extremely imbalanced datasets.
lambda	1	[0, inf)	Increasing prevents overfitting.	L2 regularization of weights.
alpha	0	[0, inf)	Increasing prevents overfitting.	L1 regularization of weights.
missing	None	(-inf, inf)	Finds optimal null values.	Replace null values with numerical value like -999.0, then set equal to -999.0. See Chapter 5, XGBoost Unveiled.

Figure 6.2 – XGBoost hyperparameter table

Now that the key XGBoost hyperparameters have been presented, let's get to know them better by tuning them one at a time.

Applying XGBoost hyperparameters

The XGBoost hyperparameters presented in this section are frequently fine-tuned by machine learning practitioners. After a brief explanation of each hyperparameter, we will test standard variations using the **`grid_search`** function defined in the previous section.

n_estimators

Recall that **`n_estimators`** provides the number of trees in the ensemble. In the case of XGBoost, **`n_estimators`** is the number of trees trained on the residuals.

Initialize a grid search of **`n_estimators`** with the default of **`100`**, then double the number of trees through **`800`** as follows:

```
grid_search(params={'n_estimators':[100, 200, 400, 800]})
```

The output is as follows:

```
Best params: {'n_estimators': 100}
Best score: 0.78235
```

Since our dataset is small, increasing **`n_estimators`** did not produce better results. One strategy for finding an ideal value of **`n_estimators`** is discussed in the *Applying early stopping* section in this chapter.

learning_rate

`learning_rate` shrinks the weights of trees for each round of boosting. By lowering **`learning_rate`**, more trees are required to produce better scores. Lowering **`learning_rate`** prevents overfitting because the size of the weights carried forward is smaller.

A default value of **`0.3`** is used, though previous versions of scikit-learn have used **`0.1`**. Here is a starting range for **`learning_rate`** as placed inside our **`grid_search`** function:

```
grid_search(params={'learning_rate':[0.01, 0.05, 0.1, 0.2, 0.3, 0.4, 0.5]})
```

The output is as follows:

```
Best params: {'learning_rate': 0.05}
Best score: 0.79585
```

Changing the learning rate has resulted in a slight increase. As described in ***Chapter 4***, *From Gradient Boosting to XGBoost*, lowering **`learning_rate`** may be advantageous when **`n_estimators`** goes up.

max_depth

`max_depth` determines the length of the tree, equivalent to the number of rounds of splitting. Limiting **`max_depth`** prevents overfitting because the individual trees can only grow as far as **`max_depth`** allows. XGBoost provides a default **`max_depth`** value of six:

```
grid_search(params={'max_depth':[2, 3, 5, 6, 8]})
```

The output is as follows:

```
Best params: {'max_depth': 2}
Best score: 0.79902
```

Changing **`max_depth`** from **`6`** to **`2`** gave a better score. The lower value for **`max_depth`** means variance has been reduced.

gamma

Known as a **Lagrange multiplier**, **`gamma`** provides a threshold that nodes must surpass before making further splits according to the loss function. There is no upper limit to the value of **`gamma`**. The default is **`0`**, and anything over **`10`** is considered very high.
Increasing **`gamma`** results in a more conservative model:

```
grid_search(params={'gamma':[0, 0.1, 0.5, 1, 2, 5]})
```

The output is as follows:

```
Best params: {'gamma': 0.5}
Best score: 0.79574
```

Changing **`gamma`** from **`0`** to **`0.5`** has resulted in a slight improvement.

min_child_weight

`min_child_weight` refers to the minimum sum of weights required for a node to split into a child. If the sum of the weights is less than the value of **`min_child_weight`**, no further splits are made. **`min_child_weight`** reduces overfitting by increasing its value:

```
grid_search(params={'min_child_weight':[1, 2, 3, 4, 5]})
```

The output is as follows:

```
Best params: {'min_child_weight': 5}
Best score: 0.81219
```

A slight adjustment to **`min_child_weight`** gives the best results yet.

subsample

The **subsample** hyperparameter limits the percentage of training instances (rows) for each boosting round. Decreasing **subsample** from 100% reduces overfitting:

```
grid_search(params={'subsample':[0.5, 0.7, 0.8, 0.9, 1]})
```

The output is as follows:

```
Best params: {'subsample': 0.8}
Best score: 0.79579
```

The score has improved by a slight amount once again, indicating a small presence of overfitting.

colsample_bytree

Similar to **subsample**, **colsample_bytree** randomly selects particular columns according to the given percentage. **colsample_bytree** is useful for limiting the influence of columns and reducing variance. Note that **colsample_bytree** takes a percentage as input, not the number of columns:

```
grid_search(params={'colsample_bytree':[0.5, 0.7, 0.8, 0.9, 1]})
```

The output is as follows:

```
Best params: {'colsample_bytree': 0.7}
Best score: 0.79902
```

Gains here are minimal at best. You are encouraged to try **colsample_bylevel** and **colsample_bynode** on your own. **colsample_bylevel** randomly selects columns for each tree depth, and **colsample_bynode** randomly selects columns when evaluating each tree split.

Fine-tuning hyperparameters is an art and a science. As with both disciplines, varied approaches work. Next, we will look into early stopping as a specific strategy for fine-tuning **n_estimators**.

Applying early stopping

Early stopping is a general method to limit the number of training rounds in iterative machine learning algorithms. In this section, we look at **eval_set**, **eval_metric**, and **early_stopping_rounds** to apply early stopping.

What is early stopping?

Early stopping provides a limit to the number of rounds that iterative machine learning algorithms train on. Instead of predefining the number of training rounds, early stopping allows training to continue until *n* consecutive rounds fail to produce any gains, where *n* is a number decided by the user.

It doesn't make sense to only choose multiples of 100 when looking for **`n_estimators`**. It's possible that the best value is 737 instead of 700. Finding a value this precise manually can be tiring, especially when hyperparameter adjustments may require changes down the road.

With XGBoost, a score may be determined after each boosting round. Although scores go up and down, eventually scores will level off or move in the wrong direction.

A peak score is reached when all subsequent scores fail to provide any gains. You determine the peak after 10, 20, or 100 training rounds fail to improve upon the score. You choose the number of rounds.

In early stopping, it's important to give the model sufficient time to fail. If the model stops too early, say, after five rounds of no improvement, the model may miss general patterns that it could pick up on later. As with deep learning, where early stopping is used frequently, gradient boosting needs sufficient time to find intricate patterns within data.

For XGBoost, **`early_stopping_rounds`** is the key parameter for applying early stopping. If **`early_stopping_rounds=10`**, the model will stop training after 10 consecutive training rounds fail to improve the model. Similarly, if **`early_stopping_rounds=100`**, training continues until 100 consecutive rounds fail to improve the model.

Now that you understand what early stopping is, let's take a look at **`eval_set`** and **`eval_metric`**.

eval_set and eval_metric

`early_stopping_rounds` is not a hyperparameter, but a strategy for optimizing the **`n_estimators`** hyperparameter.

Normally when choosing hyperparameters, a test score is given after all boosting rounds are complete. To use early stopping, we need a test score after each round.

`eval_metric` and **`eval_set`** may be used as parameters for **`.fit`** to generate test scores for each training round. **`eval_metric`** provides the scoring method, commonly **`'error'`** for classification, and **`'rmse'`** for regression. **`eval_set`** provides the test to be evaluated, commonly **`X_test`** and **`y_test`**.

The following six steps display an evaluation metric for each round of training with the default **`n_estimators=100`**:

1. Split the data into training and test sets:
```
from sklearn.model_selection import train_test_split
X_train, X_test, y_train, y_test = train_test_split(X, y, random_state=2)
```
2. Initialize the model:
```
model = XGBClassifier(booster='gbtree', objective='binary:logistic',
random_state=2)
```
3. Declare **`eval_set`**:
```
eval_set = [(X_test, y_test)]
```
4. Declare **`eval_metric`**:
```
eval_metric = 'error'
```
5. Fit the model with **`eval_metric`** and **`eval_set`**:
```
model.fit(X_train, y_train, eval_metric=eval_metric, eval_set=eval_set)
```
6. Check the final score:
```
y_pred = model.predict(X_test)
accuracy = accuracy_score(y_test, y_pred)
print("Accuracy: %.2f%%" % (accuracy * 100.0))
```

Here is the truncated output:

```
[0] validation_0-error:0.15790
[1] validation_0-error:0.10526
[2] validation_0-error:0.11842
[3] validation_0-error:0.13158
[4] validation_0-error:0.11842
…
[96] validation_0-error:0.17105
[97] validation_0-error:0.17105
[98] validation_0-error:0.17105
[99] validation_0-error:0.17105
Accuracy: 82.89%
```

Do not get too excited about the score as we have not used cross-validation. In fact, we know that **`StratifiedKFold`** cross-validation gives a mean accuracy of 78% when **`n_estimators=100`**. The disparity in scores comes from the difference in test sets.

early_stopping_rounds

`early_stopping_rounds` is an optional parameter to include with **`eval_metric`** and **`eval_set`** when fitting a model.

Let's try **`early_stopping_rounds=10`**.

The previous code is repeated with **`early_stopping_rounds=10`** added in:

```
model = XGBClassifier(booster='gbtree', objective='binary:logistic',
random_state=2)
```

```
eval_set = [(X_test, y_test)]
eval_metric='error'
model.fit(X_train, y_train, eval_metric="error", eval_set=eval_set,
early_stopping_rounds=10, verbose=True)
y_pred = model.predict(X_test)
accuracy = accuracy_score(y_test, y_pred)
print("Accuracy: %.2f%%" % (accuracy * 100.0))
```

The output is as follows:

```
[0] validation_0-error:0.15790
Will train until validation_0-error hasn't improved in 10 rounds.
[1] validation_0-error:0.10526
[2] validation_0-error:0.11842
[3] validation_0-error:0.13158
[4] validation_0-error:0.11842
[5] validation_0-error:0.14474
[6] validation_0-error:0.14474
[7] validation_0-error:0.14474
[8] validation_0-error:0.14474
[9] validation_0-error:0.14474
[10] validation_0-error:0.14474
[11] validation_0-error:0.15790
Stopping. Best iteration:
[1] validation_0-error:0.10526
Accuracy: 89.47%
```

The result may come as a surprise. Early stopping reveals that **`n_estimators=2`** gives the best result, which may be an account of the test fold.

Why only two trees? By only giving the model 10 rounds to improve upon accuracy, it's possible that patterns within the data have not yet been discovered. However, the dataset is very small, so it's possible that two boosting rounds gives the best possible result.

A more thorough approach is to use larger values, say, **`n_estimators = 5000`** and **`early_stopping_rounds=100`**.

By setting **`early_stopping_rounds=100`**, you are guaranteed to reach the default of **`100`** boosted trees presented by XGBoost.

Here is the code that gives a maximum of 5,000 trees and that will stop after 100 consecutive rounds fail to find any improvement:

```
model = XGBClassifier(random_state=2, n_estimators=5000)
eval_set = [(X_test, y_test)]
eval_metric="error"
model.fit(X_train, y_train, eval_metric=eval_metric, eval_set=eval_set,
early_stopping_rounds=100)
y_pred = model.predict(X_test)
```

```
accuracy = accuracy_score(y_test, y_pred)
print("Accuracy: %.2f%%" % (accuracy * 100.0))
```
Here is the truncated output:

```
[0] validation_0-error:0.15790
Will train until validation_0-error hasn't improved in 100 rounds.
[1] validation_0-error:0.10526
[2] validation_0-error:0.11842
[3] validation_0-error:0.13158
[4] validation_0-error:0.11842
...
[98] validation_0-error:0.17105
[99] validation_0-error:0.17105
[100] validation_0-error:0.17105
[101] validation_0-error:0.17105
Stopping. Best iteration:
[1] validation_0-error:0.10526
Accuracy: 89.47%
```
After 100 rounds of boosting, the score provided by two trees remains the best.

As a final note, consider that early stopping is particularly useful for large datasets when it's unclear how high you should aim.

Now, let's use the results from early stopping with all the hyperparameters previously tuned to generate the best possible model.

Combining hyperparameters

It's time to combine all the components of this chapter to improve upon the 78% score obtained through cross-validation.

As you know, there is no one-size-fits-all approach to hyperparameter fine-tuning. One approach is to input all hyperparameter ranges with **`RandomizedSearchCV`**. A more systematic approach is to tackle hyperparameters one at a time, using the best results for subsequent iterations. All approaches have advantages and limitations. Regardless of strategy, it's essential to try multiple variations and make adjustments when the data comes in.

One hyperparameter at a time

Using a systematic approach, we add one hyperparameter at a time, aggregating results along the way.

n_estimators

Even though the **`n_estimators`** value of **`2`** gave the best result, it's worth trying a range on the **`grid_search`** function, which uses cross-validation:

```
grid_search(params={'n_estimators':[2, 25, 50, 75, 100]})
```
The output is as follows:

```
Best params: {'n_estimators': 50}
Best score: 0.78907
```
It's no surprise that **`n_estimators=50`**, between the previous best value of 2, and the default of 100, gives the best result. Since cross-validation was not used in early stopping, the results here are different.

max_depth

The **`max_depth`** hyperparameter determines the length of each tree. Here is a nice range:

```
grid_search(params={'max_depth':[1, 2, 3, 4, 5, 6, 7, 8], 'n_estimators':[50]})
```
The output is as follows:

```
Best params: {'max_depth': 1, 'n_estimators': 50}
Best score: 0.83869
```
This is a very substanial gain. A tree with a depth of 1 is called a **decision tree stump**. We have gained four percentage points from our baseline model by adjusting just two hyperparameters.

A limitation with the approach of keeping the top values is that we may miss out on better combinations. Perhaps **`n_estimators=2`** or **`n_esimtators=100`** gives better results in conjunction with **`max_depth`**. Let's find out:

```
grid_search(params={'max_depth':[1, 2, 3, 4, 6, 7, 8], 'n_estimators':[2, 50,
100]})
```
The output is as follows:

```
Best params: {'max_depth': 1, 'n_estimators': 50}
Best score: 0.83869
```
`n_estimators=50` and **`max_depth=1`** still give the best results, so we will use them going forward, returning to our early stopping analysis later.

learning_rate

Since **`n_esimtators`** is reasonably low, adjusting **`learning_rate`** may improve results. Here is a standard range:

```
grid_search(params={'learning_rate':[0.01, 0.05, 0.1, 0.2, 0.3, 0.4, 0.5],
'max_depth':[1], 'n_estimators':[50]})
```

The output is as follows:

```
Best params: {'learning_rate': 0.3, 'max_depth': 1, 'n_estimators': 50}
Best score: 0.83869
```

This is the same score as previously obtained. Note that a **`learning_rate`** value of 0.3 is the default value provided by XGBoost.

min_child_weight

Let's see whether adjusting the sum of weights required to split into child nodes increases the score:

```
grid_search(params={'min_child_weight':[1, 2, 3, 4, 5], 'max_depth':[1],
'n_estimators':[50]})
```

The output is as follows:

```
Best params: {'max_depth': 1, 'min_child_weight': 1, 'n_estimators': 50}
Best score: 0.83869
```

In this case, the best score is the same. Note that 1 is the default for **`min_child_weight`**.

subsample

If reducing variance is beneficial, **`subsample`** may work by limiting the percentage of samples. In this case, however, there are only 303 samples to begin with, and a small number of samples makes it difficult to adjust hyperparameters to improve scores. Here is the code:

```
grid_search(params={'subsample':[0.5, 0.6, 0.7, 0.8, 0.9, 1], 'max_depth':[1],
'n_estimators':[50]})
```

The output is as follows:

```
Best params: {'max_depth': 1, 'n_estimators': 50, 'subsample': 1}
Best score: 0.83869
```

Still no gains. At this point, you may be wondering whether new gains would have continued with **`n_esimtators=2`**.

Let's find out by using a comprehensive grid search of the values used thus far.

```
grid_search(params={'subsample':[0.5, 0.6, 0.7, 0.8, 0.9, 1],
                    'min_child_weight':[1, 2, 3, 4, 5],
                    'learning_rate':[0.1, 0.2, 0.3, 0.4, 0.5],
                    'max_depth':[1, 2, 3, 4, 5],
                    'n_estimators':[2]})
```

The output is as follows:

```
Best params: {'learning_rate': 0.5, 'max_depth': 2, 'min_child_weight': 4,
'n_estimators': 2, 'subsample': 0.9}
Best score: 0.81224
```
It's not surprising that a classifier with only two trees performs worse. Even though the initial scores were better, it does not go through enough iterations for the hyperparameters to make significant adjustments.

Hyperparameter adjustments

When shifting directions with hyperparameters, **`RandomizedSearchCV`** is useful due to the extensive range of inputs.

Here is a range of hyperparameter values combining new inputs with previous knowledge. Limiting ranges with **`RandomizedSearchCV`** increases the odds of finding the best combination. Recall that **`RandomizedSearchCV`** is useful when the total number of combinations is too time-consuming for a grid search. There are 4,500 possible combinations with the following options:

```
grid_search(params={'subsample':[0.5, 0.6, 0.7, 0.8, 0.9, 1],
                    'min_child_weight':[1, 2, 3, 4, 5],
                    'learning_rate':[0.1, 0.2, 0.3, 0.4, 0.5],
                    'max_depth':[1, 2, 3, 4, 5, None],
                    'n_estimators':[2, 25, 50, 75, 100]},
                    random=True)
```
The output is as follows:

```
Best params: {'subsample': 0.6, 'n_estimators': 25, 'min_child_weight': 4,
'max_depth': 4, 'learning_rate': 0.5}
Best score: 0.82208
```
This is interesting. Different values are obtaining good results.

We use the hyperparameters from the best score going forward.

Colsample

Now, let's try **`colsample_bytree`**, **`colsample_bylevel`**, and **`colsample_bynode`**, in that order.

colsample_bytree

Let's start with **`colsample_bytree`**:

```
grid_search(params={'colsample_bytree':[0.5, 0.6, 0.7, 0.8, 0.9, 1],
'max_depth':[1], 'n_estimators':[50]})
```
The output is as follows:

```
Best params: {'colsample_bytree': 1, 'max_depth': 1, 'n_estimators': 50}
Best score: 0.83869
```
The score has not improved. Next, try **`colsample_bylevel`**.

colsample_bylevel

Use the following code to try out **`colsample_bylevel`**:

```
grid_search(params={'colsample_bylevel':[0.5, 0.6, 0.7, 0.8, 0.9,
1],'max_depth':[1], 'n_estimators':[50]})
```
The output is as follows:

```
Best params: {'colsample_bylevel': 1, 'max_depth': 1, 'n_estimators': 50}
Best score: 0.83869
```
Still no gain.

It seems that we are peaking out with the shallow dataset. Let's try a different approach. Instead of using **`colsample_bynode`** alone, let's tune all colsamples together.

colsample_bynode

Try the following code:

```
grid_search(params={'colsample_bynode':[0.5, 0.6, 0.7, 0.8, 0.9, 1],
'colsample_bylevel':[0.5, 0.6, 0.7, 0.8, 0.9, 1], 'colsample_bytree':[0.5, 0.6,
0.7, 0.8, 0.9, 1], 'max_depth':[1], 'n_estimators':[50]})
```
The output is as follows:

```
Best params: {'colsample_bylevel': 0.9, 'colsample_bynode': 0.5,
'colsample_bytree': 0.8, 'max_depth': 1, 'n_estimators': 50}
Best score: 0.84852
```
Outstanding. Working together, the colsamples have combined to deliver the highest score yet, 5 percentage points higher than the original.

gamma

The last hyperparameter that we will attempt to fine-tune is **`gamma`**. Here is a range of **`gamma`** values designed to reduce overfitting:

```
grid_search(params={'gamma':[0, 0.01, 0.05, 0.1, 0.5, 1, 2, 3],
'colsample_bylevel':[0.9], 'colsample_bytree':[0.8], 'colsample_bynode':[0.5],
'max_depth':[1], 'n_estimators':[50]})
```
The output is as follows:

```
Best params: {'colsample_bylevel': 0.9, 'colsample_bynode': 0.5,
'colsample_bytree': 0.8, 'gamma': 0, 'max_depth': 1, 'n_estimators': 50}
Best score: 0.84852
```

`gamma` remains at the default value of `0`.

Since our best score is over five percentage points higher than the original, no small feat with XGBoost, we will stop here.

Summary

In this chapter, you prepared for hyperparameter fine-tuning by establishing a baseline XGBoost model using **`StratifiedKFold`**. Then, you combined **`GridSearchCV`** and **`RandomizedSearchCV`** to form one powerful function. You learned the standard definitions, ranges, and applications of key XGBoost hyperparameters, in addition to a new technique called early stopping. You synthesized all functions, hyperparameters, and techniques to fine-tune the heart disease dataset, gaining an impressive five percentage points from the default XGBoost classifier.

XGBoost hyperparameter fine-tuning takes time to master, and you are well on your way. Fine-tuning hyperparameters is a key skill that separates machine learning experts from machine learning novices. Knowledge of XGBoost hyperparameters is not just useful, it's essential to get the most out of the machine learning models that you build.

Congratulations on completing this important chapter.

Next, we present a case study of XGBoost regression from beginning to end, highlighting the power, range, and applications of **`XGBClassifier`**.